AN IMPROVED ALGORITHM FOR CHECKING THE COLLATZ CONJECTURE FOR ALL $n < 2^N$

VIGLEIK ANGELTVEIT

Abstract. We describe a new algorithm for verifying the Collatz conjecture for all $n < 2^N$ for some fixed N. The algorithm takes less than twice as long to verify all $n < 2^{N+1}$ as it does to verify all $n < 2^N$.

1. Introduction

Let $n \geq 1$ be a natural number, and define the Collatz function $C(n)$ and the modified Collatz function $T(n)$ by

$$C(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n+1 & \text{if } n \text{ is odd} \end{cases} \qquad T(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ (3n+1)/2 & \text{if } n \text{ is odd} \end{cases}$$

The Collatz conjecture states that for any n, by applying C enough times we eventually reach the orbit $1 \mapsto 4 \mapsto 2 \mapsto 1$. Equivalently, by applying T enough times we eventually reach the orbit $1 \mapsto 2 \mapsto 1$.

A proof of the Collatz conjecture appears to be out of reach with current techniques, but considerable computer resources have been devoted to verifying the Collatz conjecture for all $n < K$ for increasingly large K. The most recent progress on the subject is described in [2], where Barina verifies convergence for all $n < 2^{71}$, and we will refer to this paper repeatedly for the current state of the art.

The goal of the current paper is to describe a more efficient algorithm for verifying the Collatz conjecture up to some K. Unlike previous algorithms, the algorithm presented in this paper becomes more efficient for higher K in the sense that the fraction of numbers in $[1, 2^{N+1}]$ that must be checked is (usually) smaller than the fraction of numbers in $[1, 2^N]$ that must be checked. Indeed, as $N \to \infty$ the fraction of numbers in $[1, 2^N]$ that must be checked approaches 0.

The key idea is to consider all n with the same k least significant bits at the same time, and then do a recursive search where we add one bit at a time.

As a further optimisation, we precompute some bitvectors and use one of them to add the A most significant binary digits in one go by looking B steps ahead, for some chosen constants A and B. This makes the tail end of the recursive search faster at the same time as it excludes more cases.

At a high level the algorithm can be described as in Algorithm 1.

Algorithm 1: a high level overview of the algorithm to verify convergence.

(1) Exclude all $n_0 < 2^{N-A}$ for which $T^k(n_0) < n_0$ for some $0 < k \leq N - A$ while simultaneously computing $T^{N-A}(n_0)$ for the remaining possible n_0, by recursively adding more bits.
(2) Use a precomputed table to exclude all $n = n_0 + a2^{N-A} < 2^N$ with $a = 0, \ldots, 2^A - 1$ for which $T^k(n) < n$ for some $N - A < k \leq N - A + B$ while also excluding all $n \equiv 2, 4, 5, 8 \mod 9$. Simultaneously compute $T^{N-A}(n)$ for the remaining values of n.
(3) Compute $T^k(n)$ for $k > T^{N-A}(n)$ until this value falls below n.

To verify convergence in a larger range than Barina we can choose $N = 72$, although we could be more ambitious and choose a larger N. In our computer implementation we chose $A = 6$ for the CPU version and $A = 10$ for the GPU version, and $B = 24$ for both versions.

In [2, Section 3.2] Barina states that using a 3^k sieve (with $k = 2$) is a time-critical operation. We implement the 3^2 sieve as a single bitwise `and` operation with one of 9 precomputed bitvectors of length 2^A, thereby reducing the number of mod 9 calculations by a large factor.

As we will explain below, our algorithm is equivalent to using a sieve of size 2^{N-A+B}, where the precomputed bitvectors have size 2^B, as well as the 3^2-sieve from [2, Section 3.2]. By this we mean that the fraction of numbers we actually need to consider is the same as the fraction one would get by using such a large sieve.

If we set $N = 72$, $A = 10$, and $B = 24$, this means our algorithm is equivalent to using a sieve of size 2^{86}. Moreover, step (2) computes $T^{N-A}(n)$, which saves time when checking those numbers that survive the initial sieves. This compares to using a sieve of size 2^{34} from [2, Table 4]. This is close to the limit of what is practical to store in memory; one advantage of our algorithm is that we avoid storing the large sieve.

1.1. **A computer implementation.** We have implemented our algorithm both for running on a CPU (in Rust) and for running on a GPU (in CUDA) using native (to the compiler) 128-bit integer types. The GPU version is, unsurprisingly, much faster. The running time appears to be significantly faster than other computer programs described in the literature. We estimate that it would take approximately 1000 years of CPU time to verify the Collatz conjecture for all $n < 2^{72}$. Running on a GPU it would take somewhere in the range 1-50 years, depending on the GPU. It can of course be split up into cases, to be processed in parallel.

We used the CPU version of our program to check all numbers up to 2^{40}, using the appropriate sieves. It explicitly checked 1 304 583 009 numbers, or about 0.12% of all natural numbers up to 2^{40}. The program took 33 seconds, which translates to about $2^{35.0}$ cases per second, on a single AMD Zen 3 core running at 4.6GHz. This is significantly faster than the 217s per 2^{40} numbers in [2, Table 7].

We used the GPU version to check all number up to 2^{50}. The program took 937s to run on a GeForce GT 1030 with 384 CUDA cores. This translates to about $2^{40.1}$ numbers per second. This GPU is obviously not state of the art: An RTX 5090 has 21 760 CUDA cores, and we can only assume our program will run substantially faster on such a GPU.

We estimate that the CPU version of our program would have to check approximately 0.023% of all natural numbers up to 2^{72} and the GPU version would have to check approximately 0.027%.

To check for correctness, we had our program output all starting values n in the appropriate range for which $T^k(n)$ reached a large value. We compared this to the table in [1], and verified that our program detects all the path records with a starting value below 2^{50}.

To make a potential distributed computation possible, we have split the problem into either 1 748 or 640 061 separate cases of (at least conjecturally) similar running time.

2. Some preliminaries

It is convenient to use the function T instead of the function C, because we get to divide by 2 exactly once each time.

Lemma 2.1 ([4])**.** *The sequence of even and odd applications of T in the k-fold composite $T^k(n)$ only depends on the last k bits of the binary representation of n.*

For example, because $3 = (11)_2$ and $11 = (1011)_2$ share the same last 3 digits we can compute $T^3(3)$ and $T^3(11)$ using the same sequence of even and odd applications of T: odd, odd, even. For 3 we get $3 \mapsto 5 \mapsto 8 \mapsto 4$ and for 11 we get $11 \mapsto 17 \mapsto 26 \mapsto 13$.

Definition 2.2. We define $f_k(n)$ to be the number of odd applications of T in $T^k(n)$.

Lemma 2.3 ([4])**.** *Suppose $n_1 = n_0 + a2^k$ for some $a \geq 0$ and that $T^k(n_0)$ has $f = f_k(n_0)$ odd applications. Then*

$$T^k(n_1) = T^k(n_0) + 3^f a.$$

For example, we can take $n_0 = 3$ and $n_1 = 3 + 1 \cdot 2^3 = 11$. Since $T^3(n_0)$ has $f = 2$ odd applications, $T^3(11) = T^3(3) + 3^2$. This has been used to implement a lookup-table for computing $T^k(n)$ in a single step by looking at the last k bits of n, and we used this in our CPU implementation. For our GPU implementation we found that it was better to avoid constantly looking up values in a table.

We immediately get the following:

Corollary 2.4. *Suppose $T^k(n_0)$ has f odd applications. If $T^k(n_0)$ is even then*

$$T^{k+1}(n_0 + 2^k) = \frac{3(T^k(n_0) + 3^f) + 1}{2},$$

and if $T^k(n_0)$ is odd then

$$T^{k+1}(n_0 + 2^k) = \frac{T^k(n_0) + 3^f}{2}.$$

This lets us compute $T^B(n)$ for all $n_0 < 2^B$ inductively, by adding one binary digit at a time starting from the least significant digit. We keep track of the number f of odd applications of T, and use a precomputed table of powers of 3.

Finally we have the following result, which lets us prune the search tree:

Theorem 2.5. *Suppose $n_1 = n_0 + a2^k$ for some $a \geq 0$ and that $T^k(n_0) < n_0$. Then $T^k(n_1) < n_1$ as well.*

This is useful because if $T^k(n_0) < n_0$ and n' converges to the cycle containing 1 for all $n' < n_0$ then n_0 must also converge to this cycle. The same reasoning applies to n_1. This means that if $T^k(n_0) < n_0$ we can exclude any natural number with the same last k binary digits as n_0.

A limited version of this idea has been used before. For example, in [3] the authors used this idea to build a sieve of size 2^k for k as large as 37, and in [2] the author uses this idea to build a sieve of size 2^{34} (for use on a CPU) or size 2^{24} (for use on a GPU). Using the above idea is equivalent to using a sieve of size 2^B, but without having to store the sieve.

We use this algorithm for the least significant $B = N - A$ bits of n, and the method from the next section for the last A bits, for some A between 6 and 10.

2.1. **The mod 9 sieve.** We refer to [2, Section 3.2]. Because $T(2n+1) = 3n+2$, any $n \equiv 2 \mod 3$ does not have to be considered because it joins the path of a smaller number. A similar argument shows that any $n \equiv 4 \mod 9$ does not have to be considered. Together this removes 4/9 of all the cases. As discussed in op. cit., using a mod 3^k sieve for larger k has limited additional benefit, so we stick with a mod 9 sieve.

In Step (2) of Algorithm 1, we use a precomputed bitvector to indicate for which a the mod 9 value of $n = n_0 + a2^{N-A}$ is allowed, for $a = 0, \ldots, 2^A - 1$.

3. A rational approximation to $\frac{\ln(3)}{\ln(2)}$ and a precomputed bitvector

In Step (2) of Algorithm 1, we would like to examine the last B bits of $T^{N-A}(n)$ to check if $T^k(n)$ goes below n for some $N - A < k \leq N - A + B$. These bits determine the sequence of even and odd applications of T, starting from $T^{N-A}(n)$. As it turns out we only need to know $f_{N-A}(n)$ and the last B bits of $T^{N-A}(n)$ to predict if this happens.

We have $\frac{65}{41} > \frac{\ln(3)}{\ln(2)}$, with $\frac{65}{41} - \frac{\ln 3}{\ln 2} = 0.000403\ldots$. It follows that 3^{41} is slightly smaller than 2^{65}, with $\frac{3^{41}}{2^{65}} = 0.9886\ldots$.

Theorem 3.1. *Suppose $T^k(n)$ has $f = f_k(n)$ odd applications and $65f \leq 41k$. Suppose also that $T^m(n) \geq 1193$ for all $0 \leq m \leq k$. Then $T^k(n) < n$.*

Proof. Each odd application increases the number by a factor of at most $\frac{3 \cdot 1193 + 1}{2 \cdot 1193}$, so $T^k(n_0)$ increases by a factor of at most $(\frac{3 \cdot 1193 + 1}{1193})^f \cdot 2^{-k} < 1$.

The requirement of $T^m(n) \geq 1193$ for all $0 \leq m \leq k$ can be relaxed, but we leave the details to the reader since we do not need a better bound. □

We will take it for granted that the Collatz Conjecture has been verified for all $n \leq 1193$, so this means that for a given n we can abort the search as soon as we have computed $T^k(n)$ and we find that $65f_k(n) \leq 41k$. In fact we do not even need to compute $T^k(n)$ if we can compute $f_k(n)$.

3.1. **Some bitvectors.** For each $m_0 \in \{0, 1, \ldots, 2^B - 1\}$ we compute the sequence of even and odd applications in $T^B(m_0)$.

Definition 3.2. Define

$$\mathrm{dip}(m_0) = \max_{k \in \{0,\ldots,B\}} 41k - 65f_k(m_0).$$

We define $\mathrm{dip}(m)$ the same way for any m sharing the same last B bits as m_0.

Fraction of bits set for $f^{(0)}$	0.03416311740875244
Fraction of bits set for $f^{(1)}$	0.12818849086761475
Fraction of bits set for $f^{(2)}$	0.2727710008621216
Fraction of bits set for $f^{(3)}$	0.451246976852417
Fraction of bits set for $f^{(4)}$	0.6307127475738525
Fraction of bits set for $f^{(5)}$	0.781611442565918
Fraction of bits set for $f^{(6)}$	0.8880811929702759
Fraction of bits set for $f^{(7)}$	0.9510104656219482

FIGURE 3.1. The fraction of bits set in each bit vector for $N = 72$, $A = 6$ and $B = 24$ for $0 \le i \le 7$.

The significance of $\mathrm{dip}(m)$ is that for any m for which $T^k(m) \ge 1193$ for all $0 \le k \le B$, $T^k(m)$ dips below $2^{-\mathrm{dip}(m_0)/41}m$ for some $0 \le k \le B$.

Given some n and a calculation of $m = T^{N-A}(n)$, we can use $\mathrm{dip}(m)$ to check if $T^k(n) < n$ for some $N - A < k \le N - A + B$. If so, the path of n joins the path of some $n' < n$ and does not need to be considered.

For a fixed f, suppose $f_{N-A}(n) = f$. Then we must have $65f > 41(N - A)$. We will consider $f^{(0)} = \lceil \frac{41(N-A)+1}{65} \rceil$ and $f^{(i)} = f^{(0)} + i$ for $i \ge 1$.

For example, suppose $N = 72$ and $A = 6$ so $N - 6 = 66$. Then $f^{(0)}, \ldots, f^{(7)} = 42, \ldots, 49$.

In this example, if n is so that $m = T^{66}(n)$ and $f_{66}(n) = 42$ it follows that if $\mathrm{dip}(m) \ge 65 \cdot 42 - 66 \cdot 41 = 24$ then $T^k(n) < n$ for some $k \le 66 + B$. We prepare one bitvector for each $f^{(i)}$ accordingly, up to some maximum value of i. If i is large, sieving has almost no effect and there are very few cases to consider, so we chose to prepare 16 such bitvectors. We omit this sieve for $i \ge 16$ and run the very small number of such cases on the CPU instead of the GPU. See Figure 3.1 for how effective each of these sieves is in an example..

To speed up the computation, we do not order the bits in the bitvector for $f^{(i)}$ linearly according to the last 24 bits in m. Instead, position s corresponds to $s3^{f_{N-A}(n)} \mod 2^B$. We do this so that the entry for $T^{N-A}(n)$ and $T^{N-A}(n+2^{N-A})$ are consecutive. Hence the entries for $T^{N-A}(n + a2^{N-A})$ for $a = 0, 1, \ldots, 2^A - 1$ occupy 2^A adjacent entries in the bitvector.

Together with the bitvectors for the mod 9 sieve discusse in Subsection 2.1, this lets us perform a single bitwise `and` operation to find those a for which $n + a2^{N-A}$ survives both sieves.

3.2. **Some empirical evidence.** In Figure 3.2 we record how many numbers need to be checked for each N for $N = 40, \ldots, 48$ using this method. Here $A = 6$ and $B = 24$.

Sometimes the percentage does not dip when going from N to $N + 1$. This is because if N is such that for n large $T^{N-A+B}(n)$ is either less than n or more $2n$ then we are not going to get any extra sieving by going from $T^{N-A+B}(n)$ to $T^{N-A+B+1}(n)$. This phenomenon can also be seen in some of the references. For example, in [4, Table 1] this corresponds to the fact that the number of "closed nodes" is 0 for some values of k (the k in that paper corresponds to $N - A + B$). From this data it looks like considering one additional bit leads to about 1.9 times as

N	cases to check	percentage to check
40	1304578225	0.1187%
41	2348308001	0.1068%
42	4696866511	0.1068%
43	8776999506	0.0998%
44	15669779158	0.0891%
45	31336736878	0.0891%
46	58370902281	0.0830%
47	116749573650	0.0830%
48	220530627195	0.0783%

FIGURE 3.2. The percentage of all numbers to check for $N = 40, \ldots, 48$.

many cases. If the trend line holds, that means we have to consider approximately 0.023% of all numbers up to 2^N when $N = 72$ and $A = 6$, or 0.027% when $A = 10$.

To support this conclusion, we ran a small simulation. For each k, we picked 100 000 000 random numbers between 0 and 2^k, and checked how many of them would survive doing the sequence of even and odd applications of T dictated by the digits of the random number. After multiplying by 5/9 to account for the mod 9 sieve, this predicted that 0.1184% of all numbers up to 2^N would survive when $N = 40$ and 0.023% would survive when $N = 72$.

We can compare this to [3, Table 3], where the authors conclude that using a size 2^{37} sieve leaves 0.70% of numbers to be considered. Hence we get an improvement of about a factor of 30, or a factor of about 17 if we add mod 9 sieving to the algorithm in [3].

4. Conclusion

We claim that the algorithm presented in this paper is much faster than other algorithms discussed in the literature, both because we end up with fewer cases to consider and because much of the work can be done very effectively using bitwise operations.

There current computer program can almost certainly be optimised, but it is already orders of magnitude faster than older programs.

The obvious next step is to find some GPU computing power, and verify convergence for $n < 2^N$ for some large N.

References

[1] David Barina. Path records. `https://pcbarina.fit.vutbr.cz/path-records.htm`. Accessed: 2025-12-05.

[2] David Barina. Improved verification limit for the convergence of the Collatz conjecture. *J Supercomput*, 81, 2025.

[3] Takumi Honda, Yasuaki Ito, and Koji Nakano. Gpu-accelerated exhaustive verification of the collatz conjecture. *International Journal of Networking and Computing*, 7(1):69–85, 2017.

[4] Tomás Oliveira e Silva. Maximum excursion and stopping time record-holders for the $3x + 1$ problem: computational results. *Math. Comp.*, 68(225):371–384, 1999.

Mathematical Sciences Institute, Australian National University, Canberra, ACT 0200, Australia